

Binary Numbers

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Recall

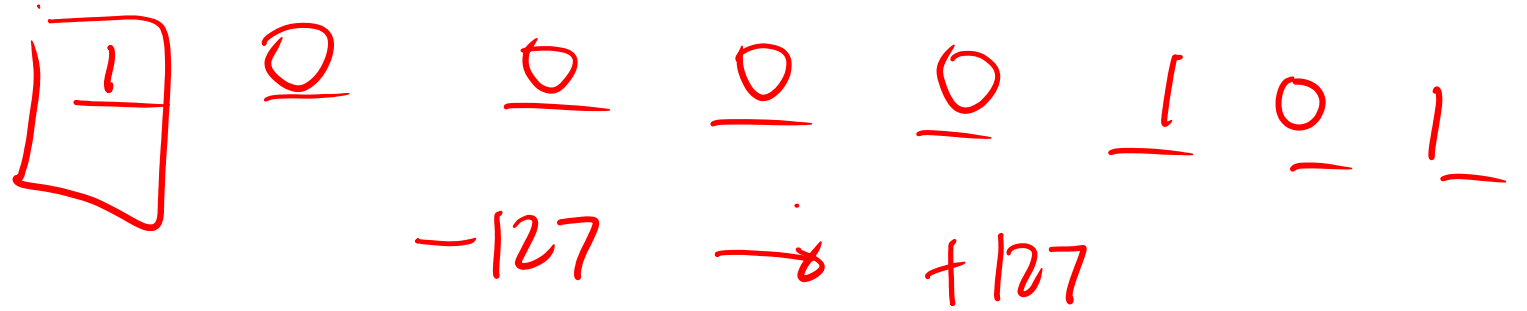
- The binary number system only has two digits: **0** and **1**
- The term **bit** is from binary digit

Discussion

$$2^8 \quad \overline{2} \quad \overline{2} \quad \overline{2} \quad \overline{2} \quad \overline{2} \quad \overline{2} \quad \overline{2} \quad \overline{2}$$

- Often, computers deal with groups of 8 bits, referred to as a **byte**
- Therefore, the smallest value of a byte is 0 while the largest is 255
- We can represent 256 different positive numbers
- How about negative numbers?

Signed Integers

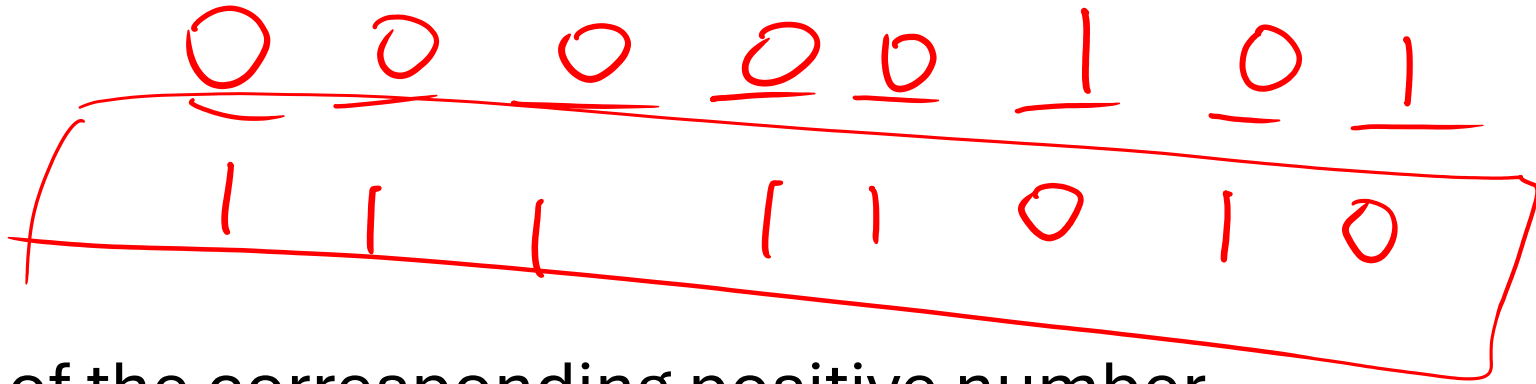


- Interpretation is determined by the hardware; determine the width
- One approach is to reserve one bit to serve as a **sign bit**
- The most significant bit is set to 0 if positive while 1 for negative
- For example, +5 is 00000101 while -5 is 10000101

Discussion

- One issue is the two representations for zero: **+0** and **-0**

1's Complement



- Simply invert all the bits of the corresponding positive number
- Positive numbers are stored as usual
- Negative numbers have all the bits of the positive number flipped
- For example, **+5** is **00000101** while **-5** is **11111010**

Notes

- Like the sign bit approach, the bit width must be defined
- Notice how there are two representations for zero: **+0** and **-0**
- For example, **+5** is **00000101** while **-5** is **11111011**

2's Complement

$$\begin{array}{r} +1 \\ \hline 10 \end{array}$$

- Common method of representing signed integers
- Perform 1's complement then add **1** to the least significant bit

$$\begin{array}{cccccccc} \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{1} & \underline{0} & \underline{1} \\ 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ + & & & & & & & 1 \\ \hline 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \end{array}$$

8

Notes /1

-8

- A negative number is simply the 2's complement of the corresponding positive number (invert bits then add 1 to the LSB)
- The MSB implies the sign: 0 for non-negative and 1 for negative

Handwritten binary addition showing the 2's complement of 8 to get -8:

	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	.	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>
	1	1	1	1	.	0	1	1	1
+									1
	1	1	1	1		1	0	0	0

The result is 1111 1000, which is the 2's complement of 8, representing -8.

Notes /2

- In two's complement, there is only one representation for 0
- Therefore, we gained an extra slot
- Given a **byte**, the range of representable values is **-128 to +127**

Bitwise Logical Operators

In C, there are four

& (AND)

| (OR)

^ (XOR)

~ (NOT)

~~28~~

||

caret

!

Bitwise And Operator

Perform a bit-by-bit comparison between the two operands

$$\underline{41}_{10} = 101001_2$$

$$\underline{54}_{10} = 110110_2$$

$$41_{10} \ \& \ 54_{10} = 100000_2 = 32_{10}$$

Bitwise Or Operator

Perform a bit-by-bit comparison between the two operands

$$\begin{array}{l} \text{41}_{10} = 101001_2 \\ \text{54}_{10} = 110110_2 \\ \hline \text{41}_{10} \mid \text{54}_{10} = 111111_2 = 63_{10} \end{array}$$

Handwritten red annotations: Above the first operand, six vertical lines mark the bits. Below the second operand, six vertical lines mark the bits, with a checkmark to the left. The final result, 63₁₀, is circled in red.

Bitwise Xor Operator

Perform a bit-by-bit comparison between the two operands

$$41_{10} = 101001_2$$

$$54_{10} = 110110_2$$

$$41_{10} \wedge 54_{10} = 011111_2 = 31_{10}$$

Bitwise Not Operator

Flips every bit in a value. Its result depends on the assumed **bit width**, so keep that in mind when using it.

[illegible]

[illegible]

Notes

- Interestingly, we can notice one pattern
- To obtain the negative version of an integer **n** , we apply **$\sim (n-1)$**

Bit Shifting Operators

In C, there are two

<< Left shift

>> Right shift

Left Shift

Shifts bits to the left

`var << n` will multiply **`var`** by **`2n`**

Practice /1

```
int val = 43;  
int result = val << 2;
```

val = 43

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

result = 172

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Practice /2

2^n

```
int val = 1;
```

```
int result = val << 1;
```

result = result << 1;

val = 1

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

result = 2

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Right Shift

Shifts bits to the right

`var >> n` will divide **`var`** by 2^n if **`var`** is nonnegative

Practice /1

```
int val = 43;  
int result = val >> 2;
```

logical

arithmetic

val = 43

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

result = 10

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Practice /2

```
int val = 16;  
int result = val >> 1;
```

val = 16

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

16

result = 8

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

8

Notes

- There are two types of terminology associated with the right shift, namely the **logical shift** and the **arithmetic shift**
- Logical shift simply fills the left with 0s regardless of sign
- Arithmetic shift fills the left with MSB (sign bit) to preserve the sign

Scenario

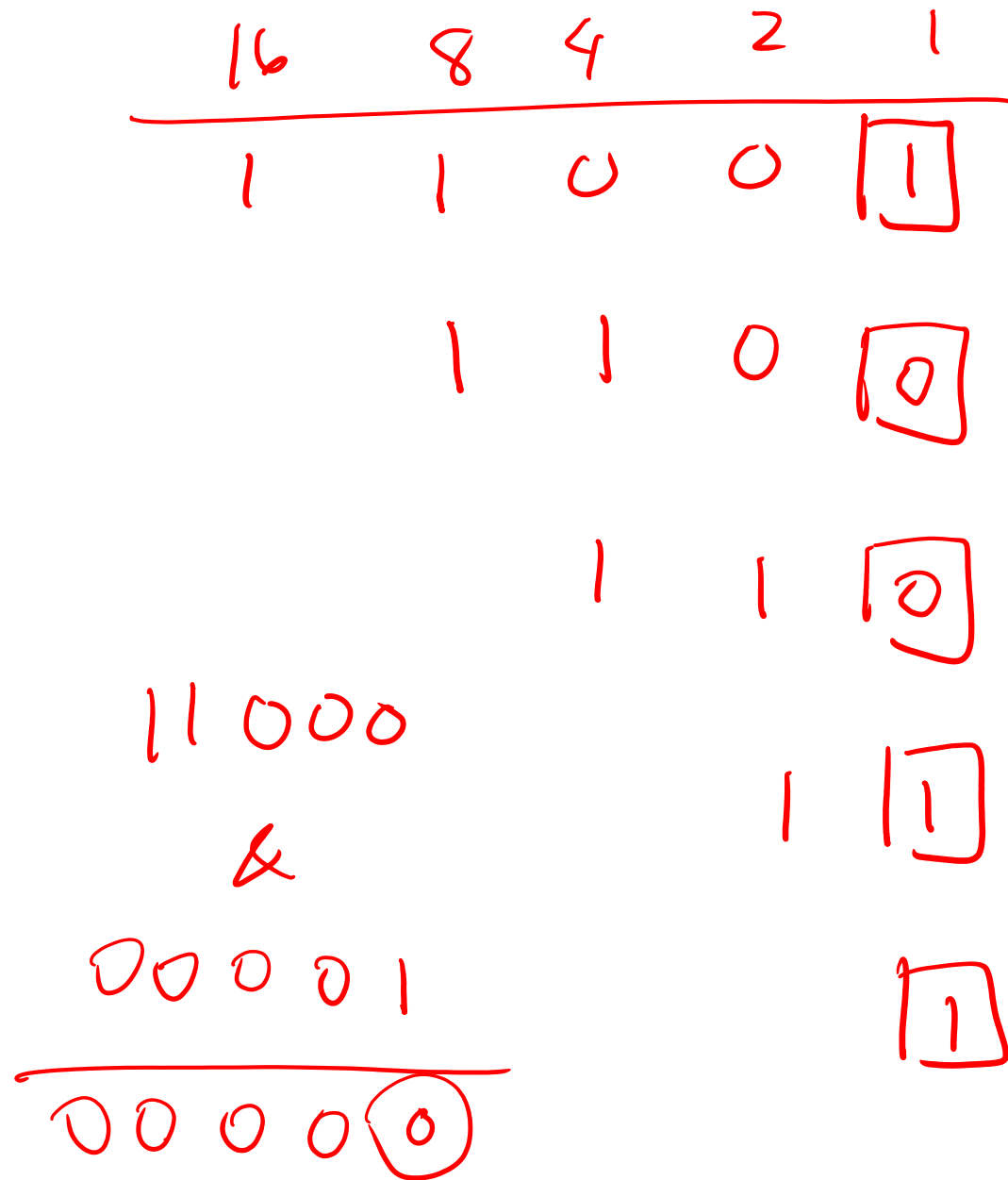
Given an integer in C, print the number of **0**s and **1**s in its binary representation.

Discussion

- Essentially, the task involves traversing each of a binary string
- In C, an integer typically uses around 4 bytes or **`sizeof(int)`**
- Think of it as a combination of a shift and a bitwise AND operation

magic

```
1 #include <stdio.h>
2
3 int main(void) {
4     unsigned int num = 25;
5     int c = 0;
6     int bits = sizeof(num)*8;
7
8     for(int i = 0; i < bits; i++) {
9         if( (num & 1) == 1 )
10             c++;
11
12         num = num >> 1;
13     }
14
15     printf("%d %d\n", c, bits-c);
16
17     return 0;
18 }
```



Scenario

$$\begin{array}{rcl} 3 & \rightarrow & 011 \\ 5 & \rightarrow & \underline{101} \\ & & \checkmark \checkmark \times \end{array}$$

Given two positive integer values, count how many bit positions differ in their binary representations. Complete a function that takes two integers as input and returns this count.

```
int count_bit_difference(unsigned a, unsigned b) {  
  
  
  
  
  
  
  
  
  
}
```

Discussion

- Assuming you have two binary strings, a relevant bitwise operator for the task is XOR
- XOR is useful when you want to check if two bits are **different**
- It returns 1 if the corresponding bits of the two operands are not the same and 0 if they are the same

```
1  #include <stdio.h>
2
3  int count_bit_difference(unsigned a, unsigned b) {
4      unsigned int num = a ^ b;
5      int c = 0;
6      int bits = sizeof(num)*8;
7
8      for(int i = 0; i < bits; i++) {
9          if( (num & 1) == 1 )
10             c++;
11
12             num = num >> 1;
13     }
14
15     return c;
16 }
```

Unsigned Integers /1

- In C, you can declare an unsigned integer using the **unsigned** keyword (e.g., **unsigned int**)
- Unlike signed integers, unsigned integers cannot represent negative values
- Instead, they use the full range of bits to represent only non-negative numbers

Unsigned Integers /2

- This increases the maximum representable value
- For example, if it is a 32-bit **unsigned int**, it will be from 0 to **4,294,967,295**
- Use the **%u** format specifier with **printf()** to display **unsigned int** values

Bit Masking

- A technique for inspecting or manipulating individual bits in a binary value through various bitwise operations
- It involves the application of a bitmask, which is simply a binary number where certain bits are set to 0 or 1
- Saves space on a group of binary flags (instead of having several individual ones)

Scenario

What is the value of the bit just before the rightmost bit of a given binary string?

V2r

1	0	0	1	1	0	1	1
---	---	---	---	---	---	---	---

8

&

2

2

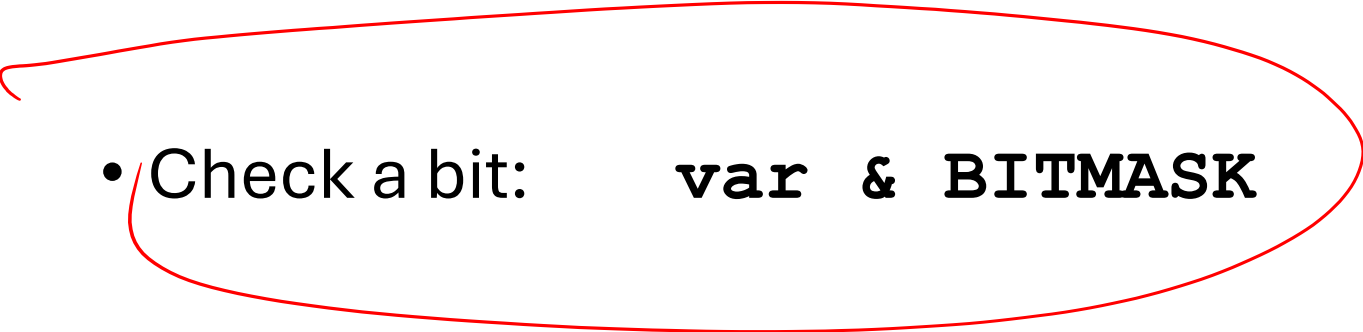
0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---

=

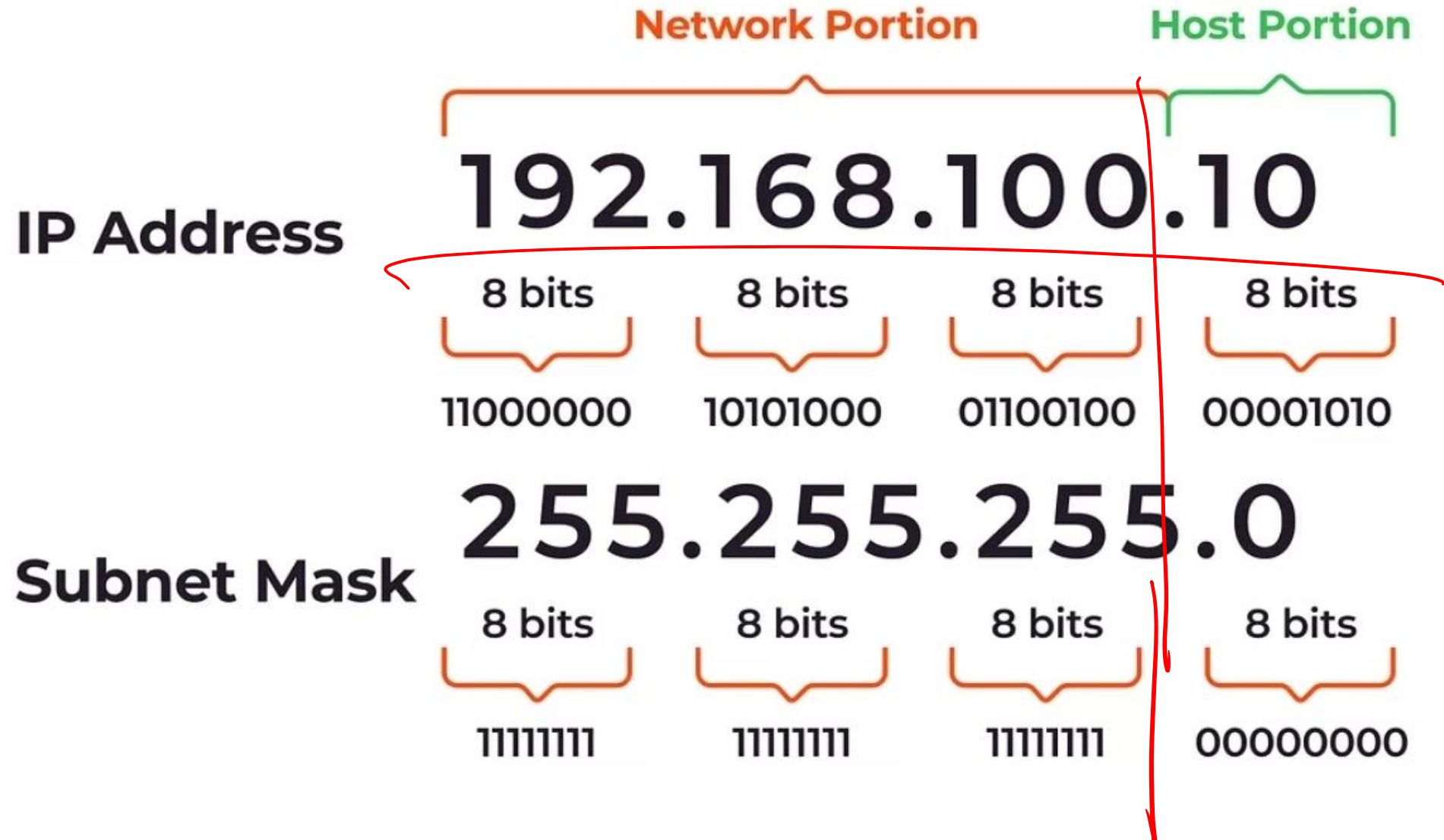
						1	
--	--	--	--	--	--	---	--



Common Operations

- Set a bit on: `var |= BITMASK`
 - Set a bit off: `var &= ~BITMASK`
 - Toggle a bit: `var ^= BITMASK`
 - Check a bit: `var & BITMASK`
- 

Binary Notation of IP Address and Subnet



Practice

4 → 100
5 → 101

Complete the function so that it returns **1** if the given number is odd, and **0** otherwise. Do not use the modulo operator.

```
int is_odd(int num) {
```

```
    return num & 1;
```

```
}
```

Your Turn!

Refer to the practice problems on Webcourses

TODO 1

Complete the function that takes the number of bits (the width) as input and returns how many binary strings of that length contain an odd number of 1s.

```
int count_odd_ones(int bits) {
```

```
int count_odd_ones(int bits) {
```

```
    int limit = 1 << bits;
    int res = 0;

    // For every possible number that can be represented
    for(int i = 0; i < limit; i++) {
        int tmp = i;
        int c = 0;

        // Count the number of 1 bits in the string
        for(int j = 0; j < limit; j++) {
            if( ( tmp & 1 ) == 1 )
                c++;

            tmp = tmp >> 1;
        }

        // Check if the rightmost is 1
        // If yes, it is an odd number
        if( c & 1 )
            res++;
    }

    return res;
}
```

```
}
```

TODO 2

Complete the function that takes a positive integer N and returns the decimal value corresponding to the rightmost 1 bit in the binary representation of N .

In other words, starting from the rightmost digit (least significant bit), find the first occurrence of a 1 and return its value in decimal form.

Example

For example, given the input 12 (binary: 1100), the rightmost 1 corresponds to 0100, so the function should return 4.

```
int rightmost_one_bit_value(int N) {
```

```
int rightmost_one_bit_value(int N) {
```

```
    int m = 1;
```

```
    // Perform a repeated bitwise AND
```

```
    // and terminate at the first 1
```

```
    while( ( N & m ) == 0 ) {
```

```
        m = m << 1;
```

```
    }
```

```
    return m;
```

```
}
```

TODO 3

Complete the function that takes a positive integer N and returns the decimal value corresponding to the leftmost 1 bit in the binary representation of N .

In other words, starting from the leftmost digit (most significant bit), find the first occurrence of a 1, and return its value in decimal form.

Assume that the integer is represented using 32 bits.

Example

For example, given the input 12 (binary: 1100), the leftmost 1 corresponds to 1000, so the function should return 8.

```
int leftmost_one_bit_value(int N) {
```

```
int leftmost_one_bit_value(int N) {
```

```
    // The 32nd bit acts as a sign bit if we use the int data type  
    // We only need to move the 1 all the way to the 31st position  
    // So, we perform 30 left shifts only
```

```
    int m = 1 << 30;
```

```
    // Perform a repeated bitwise AND  
    // and terminate at the first 1
```

```
    while( ( N & m ) == 0 ) {  
        m = m >> 1;  
    }
```

```
    return m;
```

```
}
```

TODO 4

Complete the function that takes two positive integers: a number N and a pattern P , both given in decimal form. The function should return 1 if the rightmost bits of N , with the same length as the binary representation of P , exactly match the binary pattern represented by P , and 0 otherwise.

Example

For example, let $N = 13$ (binary: 1101) and $P = 5$ (binary: 101). The rightmost 3 bits of N are 101, which match the pattern, so the function should return 1.

Important: The number of bits to compare is determined by the length of P 's binary representation (ignoring leading zeros).

```
int match_pattern(int N, int pattern) {
```

```
int match_pattern(int N, int pattern) {  
  
    int mask = 1;  
    int tmp = pattern;  
  
    // Build a bitmask with length same as pattern  
    while( tmp > 0 ) {  
        mask = mask << 1;  
        tmp = tmp >> 1;  
    }  
  
    // Subtract 1 to convert to a mask of all 1s of the same length  
    mask = mask - 1;  
  
    // Perform a bitwise AND and the result must  
    // be the same as the pattern  
    return ( N & mask ) == pattern;  
  
}
```

Questions?